



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

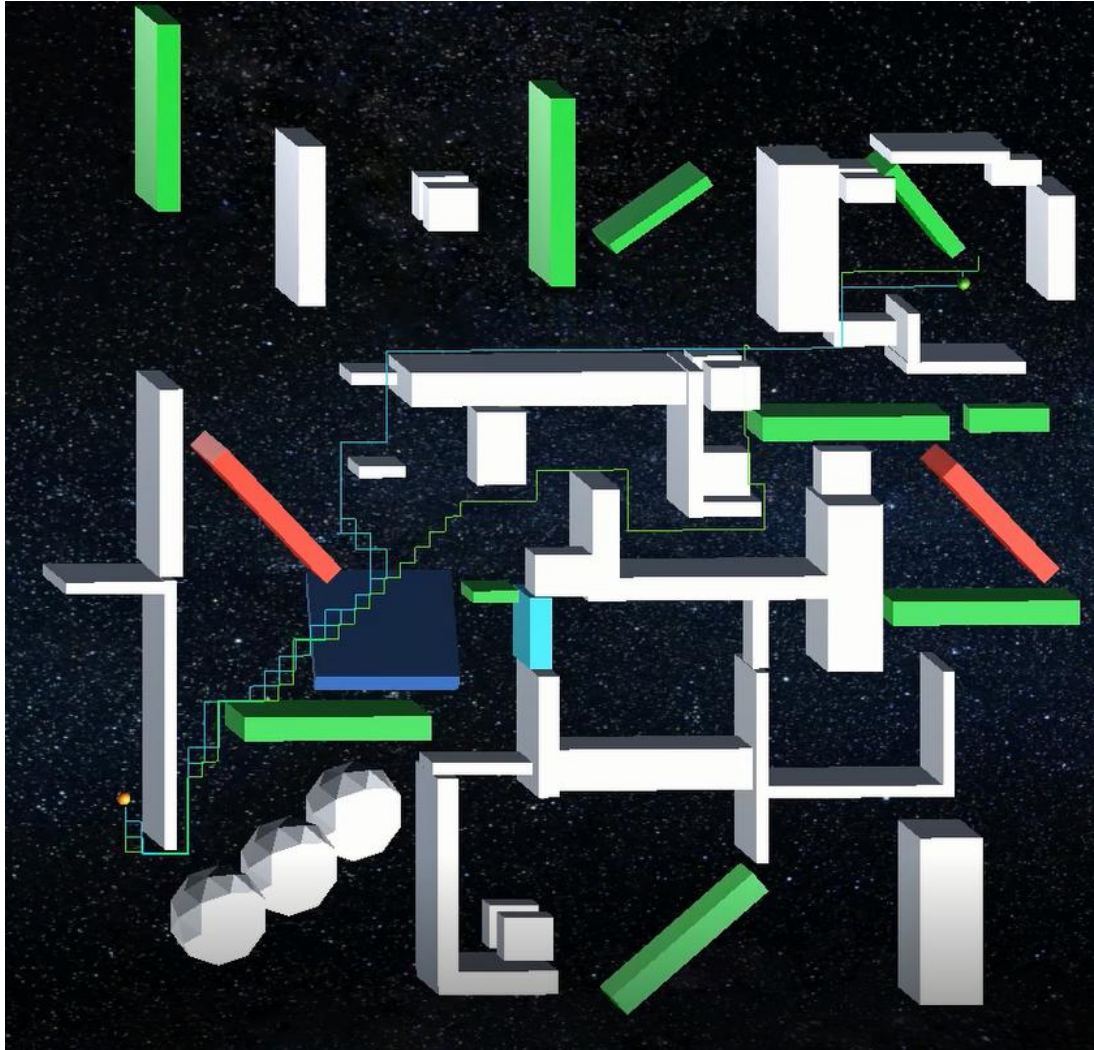
Computational Intelligence

Prof. Dr.-Ing. Sabine Schumann

Fakultät Informatik und Digitale Gesellschaft

Informierte Suche

Was erwartet Dich in diesem Kapitel?



Informierte Suchverfahren

Bewertungsfunktion

Nearest Neighbour Heuristic

Heuristiken

(gierige) Bestensuche

Hill Climbing with Backtracking

A / A* Algorithmus

Was suchen wir eigentlich?

- Gibt es überhaupt eine Lösung?
Pfad zu einer Lösung finden
- Welche Lösung ist die beste?
kürzesten Weg zu einer Lösung finden
- Lässt sich da etwas optimieren?
Qualität einer Lösung liegt in der Lösung selbst

Ziel:

- Qualitativ beste Lösung finden, d.h. Pfad mit den geringsten Kosten.
- oder
- Qualität einer Lösung liegt in der Lösung selbst und nicht der Pfad zur Lösung ist interessant.

Informierte Suche

- Bisher:
Wissen über Qualität der Zustände nicht genutzt.
- Sicher bekannt:
bisherige Kosten
- Wünschenswert:
Information über Restkosten
 - nur Schätzung möglich,
 - Verwendung von Heuristiken,
 - erprobte, aber nicht notwendigerweise optimale Methoden zur Vereinfachung von Problemen,
 - Schwierigkeit: gute bzw. beste Heuristik finden.

ευρισκειν (gr.): (er)finden, entdecken

Informierte Suche – Bewertungsfunktionen

Einfache Bewertungsfunktionen ($f(n)$):

- n : aktueller Knoten
- g : bisherige Kosten
- h : geschätzte Kosten der noch nötigen Schritte

3 Möglichkeiten:

- $f(n) = g(n)$
- $f(n) = h(n)$
- $f(n) = g(n) + h(n)$

Nearest Neighbour Heuristic

Travelling Salesman Problem

Beispiel Travelling Salesman Problem (Handelsreisenderproblem) :

- auf einer Rundreise sollen n Städte besucht werden, jede genau einmal
- dabei ist ein bestimmter Faktor zu minimieren (bspw. die Reisezeit oder kürzeste Strecke)



bei n Städten $(n-1)!$ Möglichkeiten

(exakt $\frac{(n-1)!}{2}$ exponentieller Aufwand)



Nearest Neighbour (Heuristic) – Travelling Salesman Problem

der nächste Nachbar ... ist ein falscher Freund, da er gar keine Heuristik verwendet

Bewertungsfunktion: $f(n)=g(n)$

Voraussetzung für Nearest Neighbour (NNH) ist das Aufwandsmaß g für die bisherigen Kosten der Zustandsübergänge.

NNH:

- (1) Beliebige Start-Stadt wählen.
- (2) Wähle aus den noch nicht besuchten Städten die zur aktuellen Stadt am nächsten gelegene Stadt.
- (3) Falls noch nicht alle Städte besucht wurden, gehe zu (2).



Nearest Neighbour (Heuristic)

- Kein Zurückgehen (Backtracking), um Alternativlösungen zu suchen.
- Demzufolge unvollständig und i.a. nicht optimal.
- Zeitaufwand nur $O(n^2)$.
- Nur bei Vorliegen von genauem Abstandmaß g sinnvoll.

Beispiel:

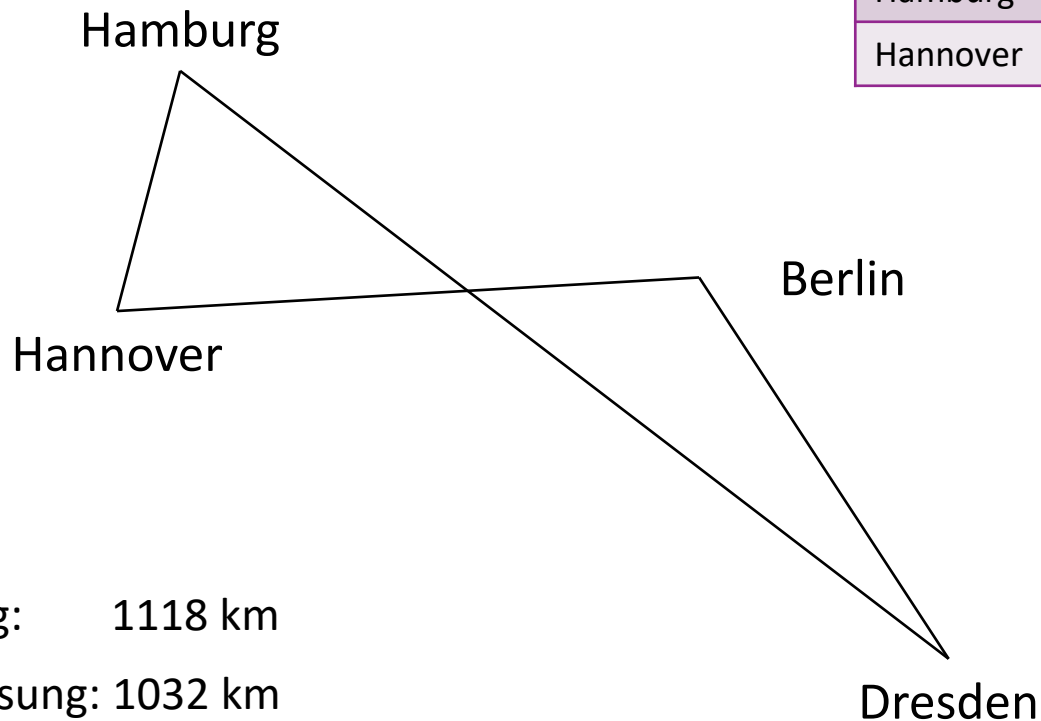
zu besuchende Städte: Berlin, Dresden, Hamburg, Hannover

Kilometer	Berlin	Dresden	Hamburg	Hannover
Berlin	0	214	279	258
Dresden	214	0	492	385
Hamburg	279	492	0	154
Hannover	258	385	154	0

Nearest Neighbour (Heuristic)

Beispiel (Fortsetzung):

Wir starten in Hamburg.

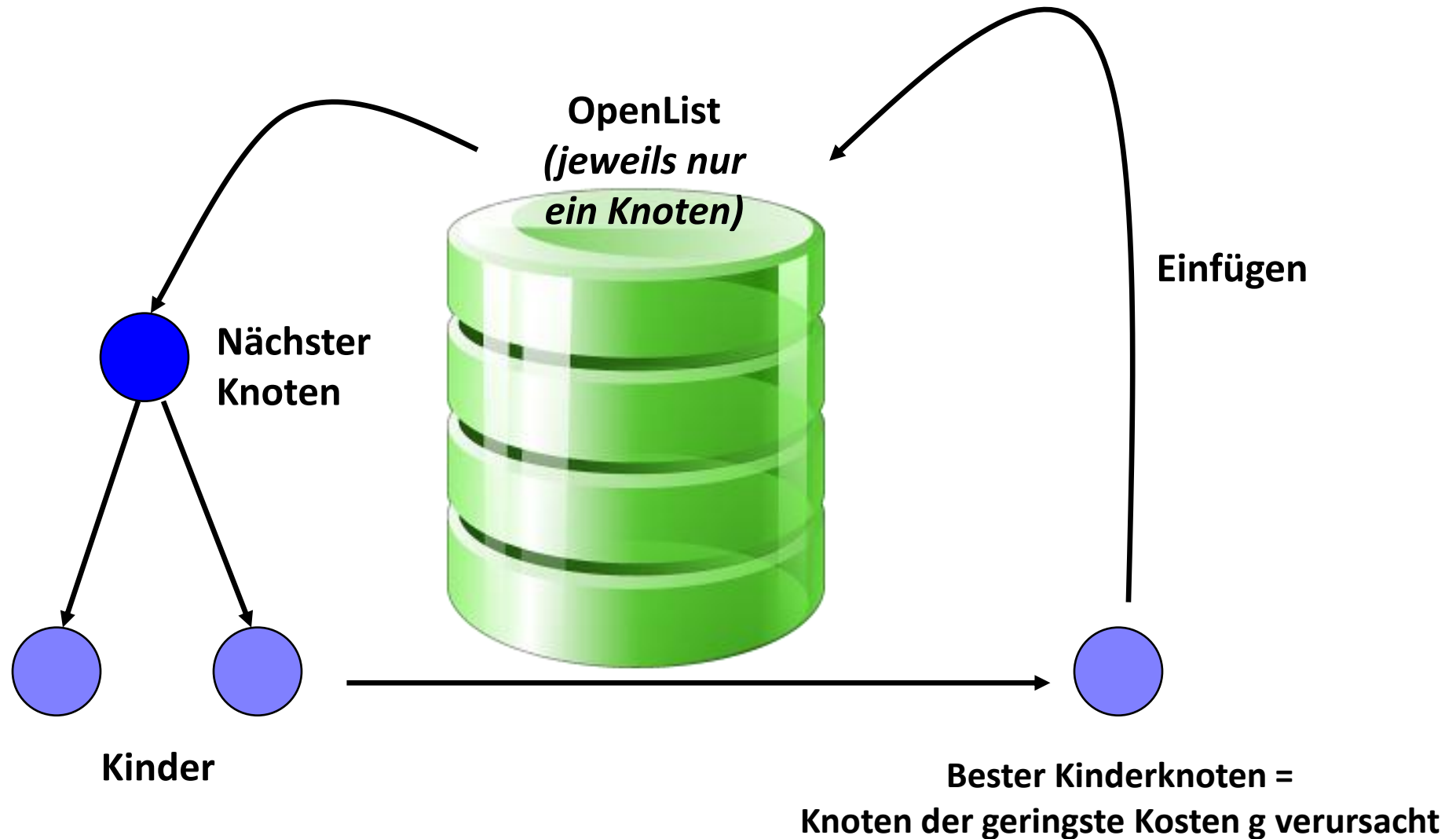


Kilometer	Berlin	Dresden	Hamburg	Hannover
Berlin	0	214	279	258
Dresden	214	0	492	385
Hamburg	279	492	0	154
Hannover	258	385	154	0

Schritt	OpenList
1	Hamburg

- NNH-Lösung: 1118 km
- optimale Lösung: 1032 km

Nearest Neighbour (Heuristic)



Übung NNH

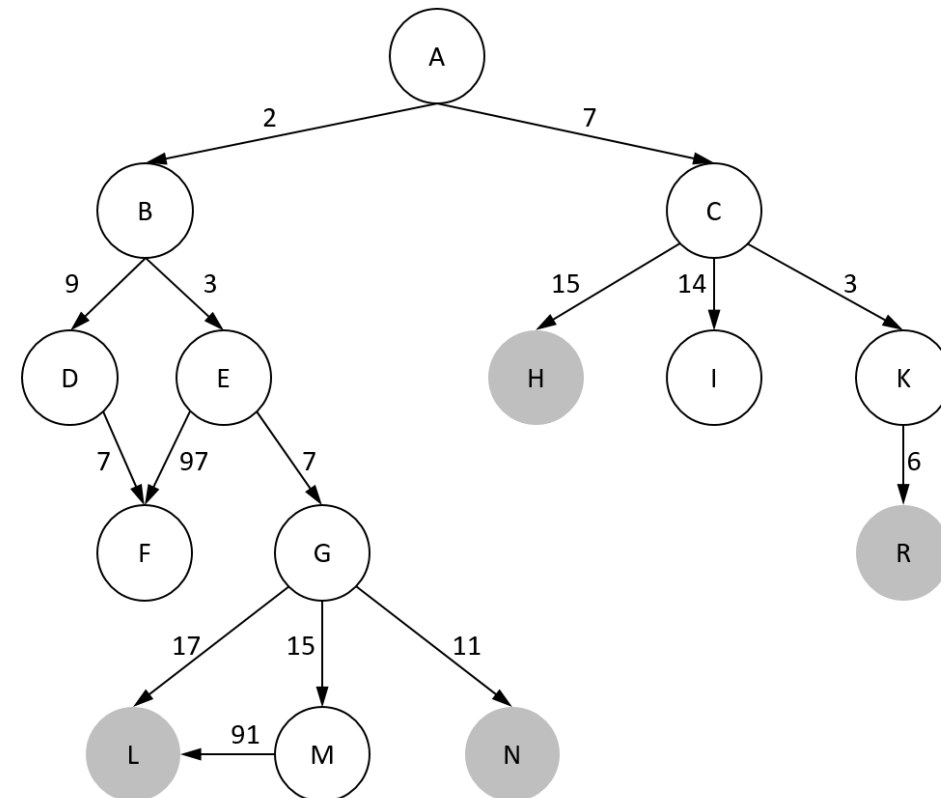
Gegeben ist folgender Suchbaum. Ermitteln Sie OpenList, Reihenfolge der besuchten Knoten und den Pfad zum ersten gefundenen Ziel (außer der Algorithmus terminiert). Ziel: ●

Vereinbarungen:

- Quantifizierung
Buchstaben, die weiter vorn im Alphabet stehen, werden bei gleicher Bewertung bevorzugt



10 Minuten



Heuristiken

Heuristische Funktion

Eine heuristische Funktion

- ist eine Funktion h , die jedem Knoten k im Suchbaum eine nicht negative Zahl $h(k)$ zuordnet. Diese Zahl gibt eine Schätzung für die Entfernung des Knotens k zum nächsten Zielknoten an. Ist k ein Zielknoten, so ist $h(k)=0$.
- ist umso besser, je stärker sie differenziert.



Wie findet man eine *nützliche* heuristische Funktion?

Beispiel 8-Puzzle:

1. $h_1(z)$ Anzahl der Spielsteine im Zustand z , die an falscher Position liegen.
Je kleiner die Zahl, desto mehr Spielsteine liegen korrekt.
2. $h_2(z)$ Summe der Entfernungen aller Spielsteine von ihrer jeweiligen Zielposition.
3. $h_3(z)$ Summe Anzahl 2x direkter Spielsteintausch

verschiedene Heuristiken für das 8-Puzzle

	h_1 : Anzahl Spielsteine auf falscher Position	h_2 : Summe der Ent- fernungen, in denen sich Spielsteine zu ihren Zielpositionen befinden	h_3 : 2x Anzahl direkter Spielsteintausch									
<table><tr><td>2</td><td>8</td><td>3</td></tr><tr><td>1</td><td>6</td><td>4</td></tr><tr><td></td><td>7</td><td>5</td></tr></table>	2	8	3	1	6	4		7	5	5	6 1+1+0+0+0+1+1+2	0
2	8	3										
1	6	4										
	7	5										
<table><tr><td>2</td><td>8</td><td>3</td></tr><tr><td>1</td><td></td><td>4</td></tr><tr><td>7</td><td>6</td><td>5</td></tr></table>	2	8	3	1		4	7	6	5	3	4 1+1+0+0+0+0+0+2	0
2	8	3										
1		4										
7	6	5										
<table><tr><td>2</td><td>5</td><td>3</td></tr><tr><td>1</td><td>6</td><td>4</td></tr><tr><td>7</td><td>8</td><td></td></tr></table>	2	5	3	1	6	4	7	8		5	8 1+1+0+0+3+1+0+2	0
2	5	3										
1	6	4										
7	8											

Ziel

1	2	3
8		4
7	6	5

Suchalgorithmen mit $f(n) = h(n)$

- Bestensuche (best-first search)
- Hill Climbing (Bergsteiger)
- Hill Climbing with Backtracking (Bergsteiger mit Backtracking)

Nützlich

- für bestimmte Suchprobleme zur Bestimmung von Lösungspfaden.
- wenn nicht der Pfad, sondern ein Zielzustand gesucht wird.
- für Optimierungsprobleme
($h(n)$ liefert Wert des Zustandes).

Bestensuche

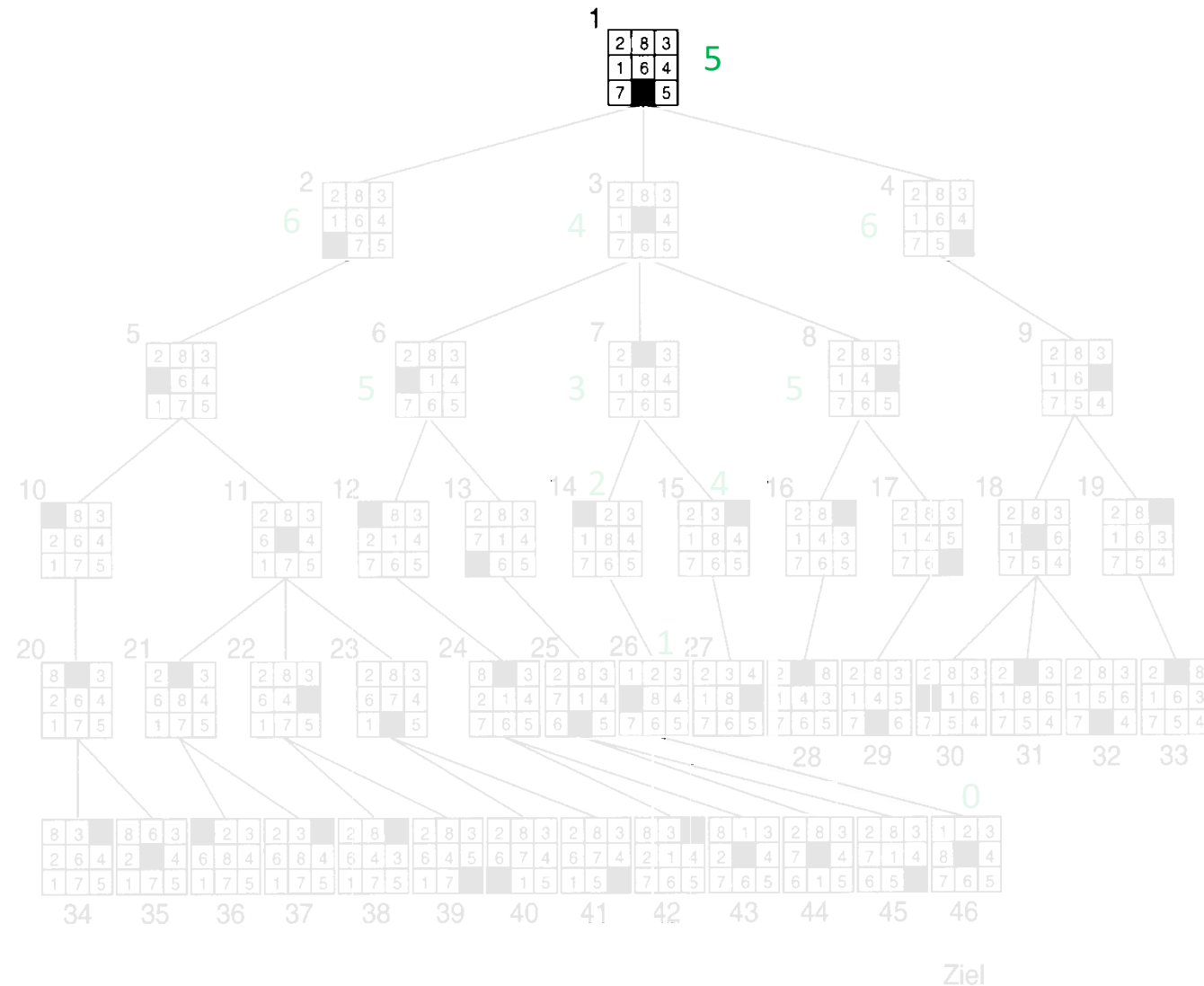
Bestensuche

- Sinn der heuristischen Funktion:
vierversprechende Knoten als nächstes zu untersuchen und alle anderen hinten an stellen.
- Sinnvoll Knoten nach ihrer Bewertung zu ordnen, die besten in der OpenList nach vorn.
- **Bestensuche** (best-first search)
verwendet demzufolge ebenfalls eine OpenList,
Einfügen der Knoten erfolgt entsprechend der heuristischen Werte.

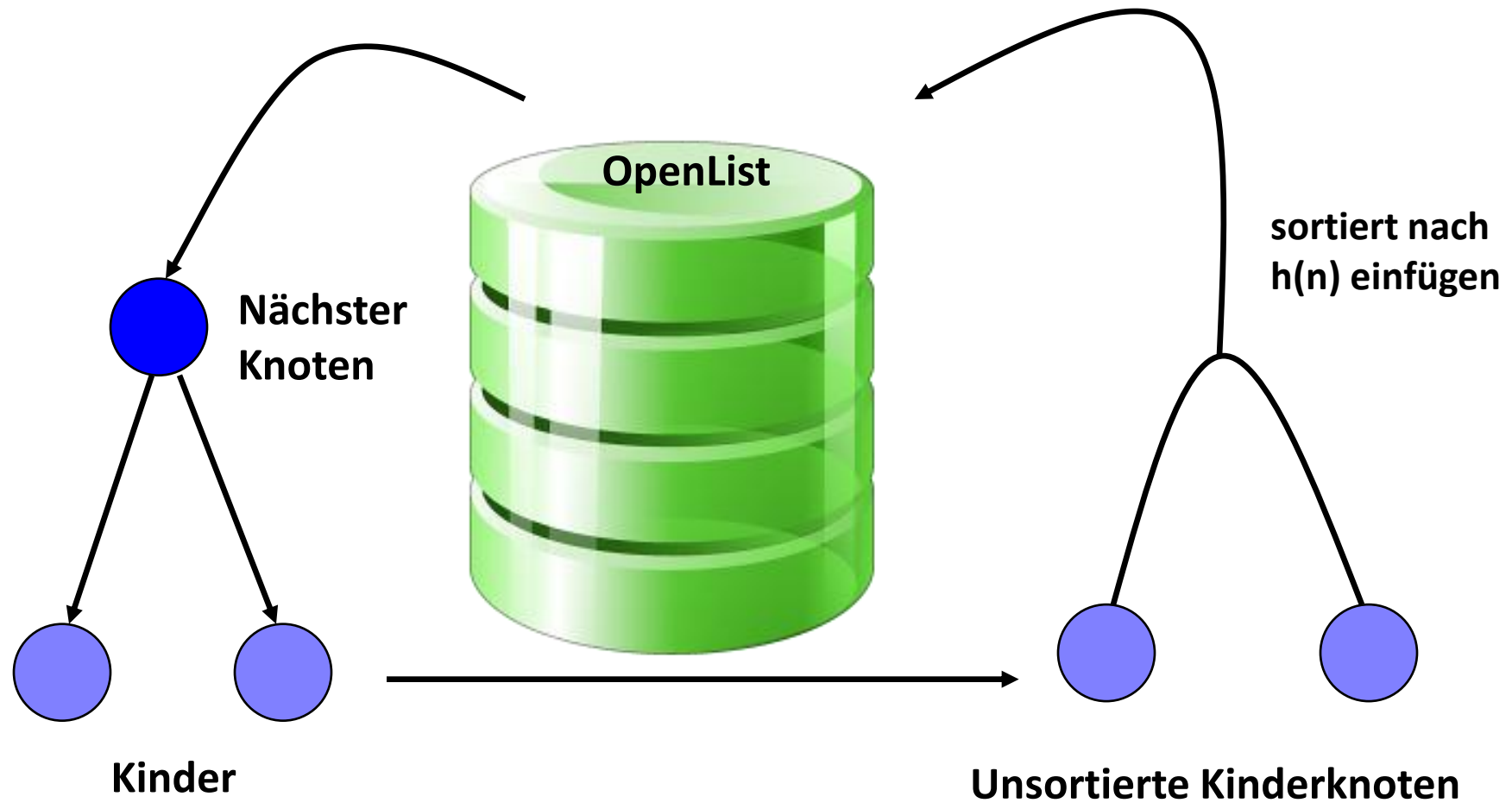
Bestensuche

Beispiel: 8-Puzzle,
heuristische Funktion $h_2(z)$

Schr.	OpenList
1	1(5)



Bestensuche

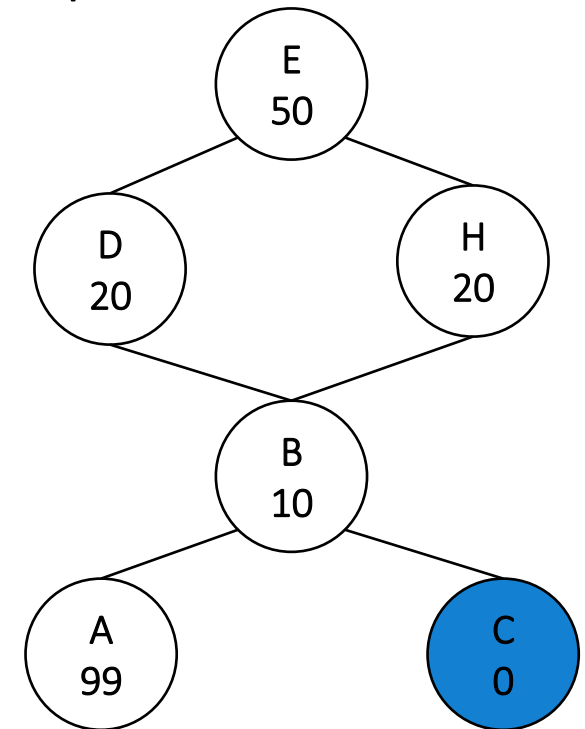


Überlegungen Bestensuche (1)

- Wie funktioniert der Algorithmus, wenn es mehrere gleich gut bewertete nächste Schritte gibt und keine Quantifizierung vorliegt?
- Wie wird sich der zurückgelegte Weg gemerkt (besuchte Knoten, Pfad)?
- Was passiert, wenn es sich nicht um einen Baum, sondern einen Graphen handelt?

Schritt	OpenList
1	E(50)
2	D(E,20), H(E,20)
3	B(DE,10), H(E,20)
4	C(BDE,0), H(E,20), A(BDE,99)
5	Ziel gefunden, H(E,20), A(BDE,99)

- Pfad zum Ziel: EDBC
- Backtracking möglich

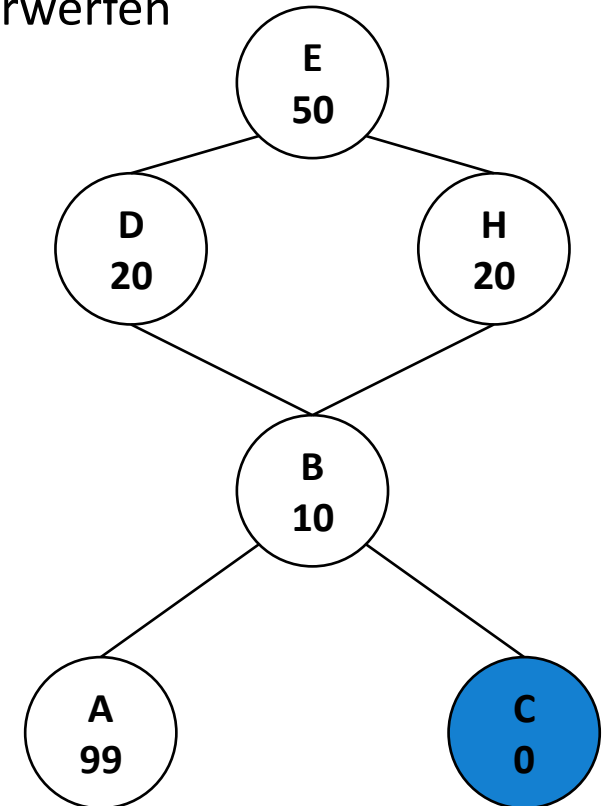


Überlegungen Bestensuche (2)

- Backtracking

Schritt	OpenList
1	E(50)
2	D(E,20), H(E,20)
3	B(DE,10), H(E,20)
4	C(BDE,0), H(E,20), A(BDE,99)
5	Ziel gefunden, H(E,20), A(BDE,99)
6	B(HE,10), A(BDE,99)
7	C(BHE,0), A(BDE,99), A(BHE,99)
8	Ziel gefunden, A(BDE,99), A(BHE,99)

Ziel und Pfad verwerfen



- weiterer Lösungsweg EHBC
- weiteres Backtracking möglich, findet jedoch kein Ziel mehr.

Übung Bestensuche

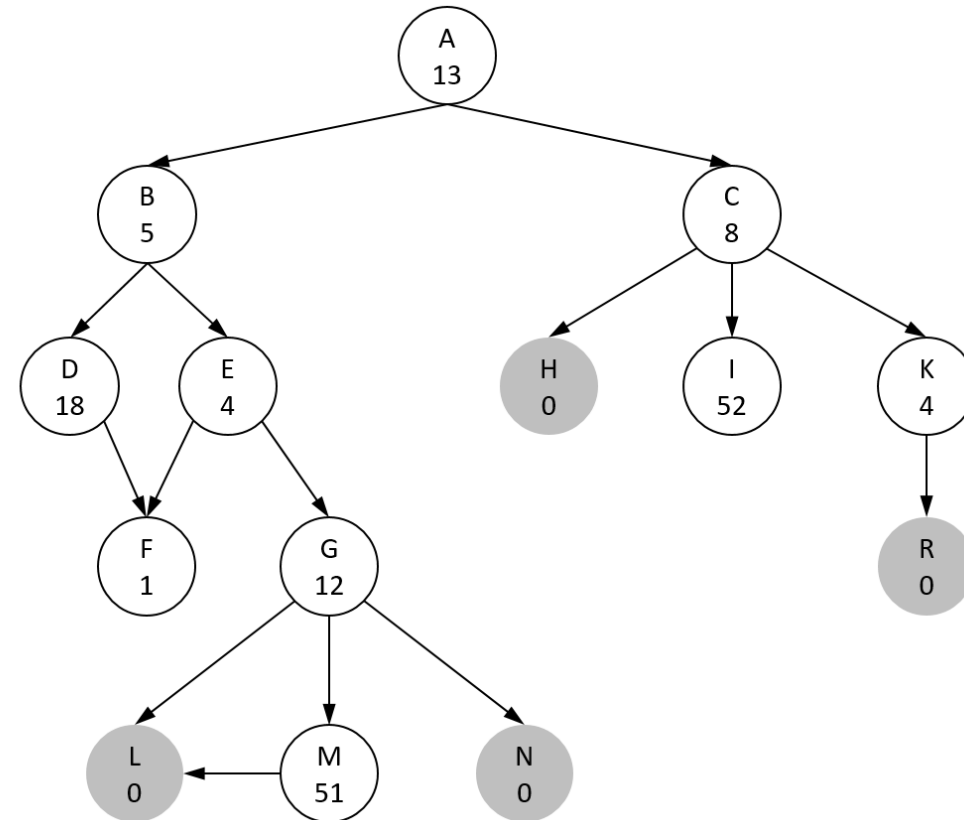
Gegeben ist folgender Suchbaum. Ermitteln Sie OpenList, Reihenfolge der besuchten Knoten und den Pfad zum ersten gefundenen Ziel (außer der Algorithmus terminiert). Ziel: ●

Vereinbarungen:

- Quantifizierung
Buchstaben, die weiter vorn im Alphabet stehen, werden bei gleicher Bewertung bevorzugt



10 Minuten



Hill Climbing (with Backtracking)

Hill Climbing – Beispielprobleme (1)

Sie erinnern sich:

Travelling Salesman Problem:

Auf einer Rundreise sollen n Städte besucht werden,
wobei ein bestimmter Faktor zu minimieren ist.



Suche nach dem optimalen Weg

Ein optimaler Weg wird nur dann gefunden, wenn jeder mögliche Weg generiert und der mit dem geringsten Wert ausgewählt wird.



$(n-1)!$ Kombinationsmöglichkeiten!!!



Hill Climbing – Beispielprobleme (2)

Travelling Salesman Problem:

Selbst wenn ein Rechner pro Weg nur 0.0001 Sekunden plant, dauert es bei 15! über 4 Jahre, den optimalen Weg zu finden. (Kombinatorische Explosion)

n	n^2	2^n	$(n-1)!$
1	1	2	1
2	4	4	1
3	9	8	2
4	16	16	6
5	25	32	24
7	49	128	720
10	100	1024	362880
12	144	4096	39916800
15	225	32768	87178291200
16	256	65536	1307674368000

Hill Climbing – Beispielprobleme (3)

Rucksack befüllen:

Der Rucksack kann maximal mit 20kg befüllt werden. Die Objekte, die eingepackt werden können, haben einen Wert und ein Gewicht. Ziel ist es, den Rucksack so zu füllen, dass die enthaltenen Objekte einen maximalen Gesamtwert haben.

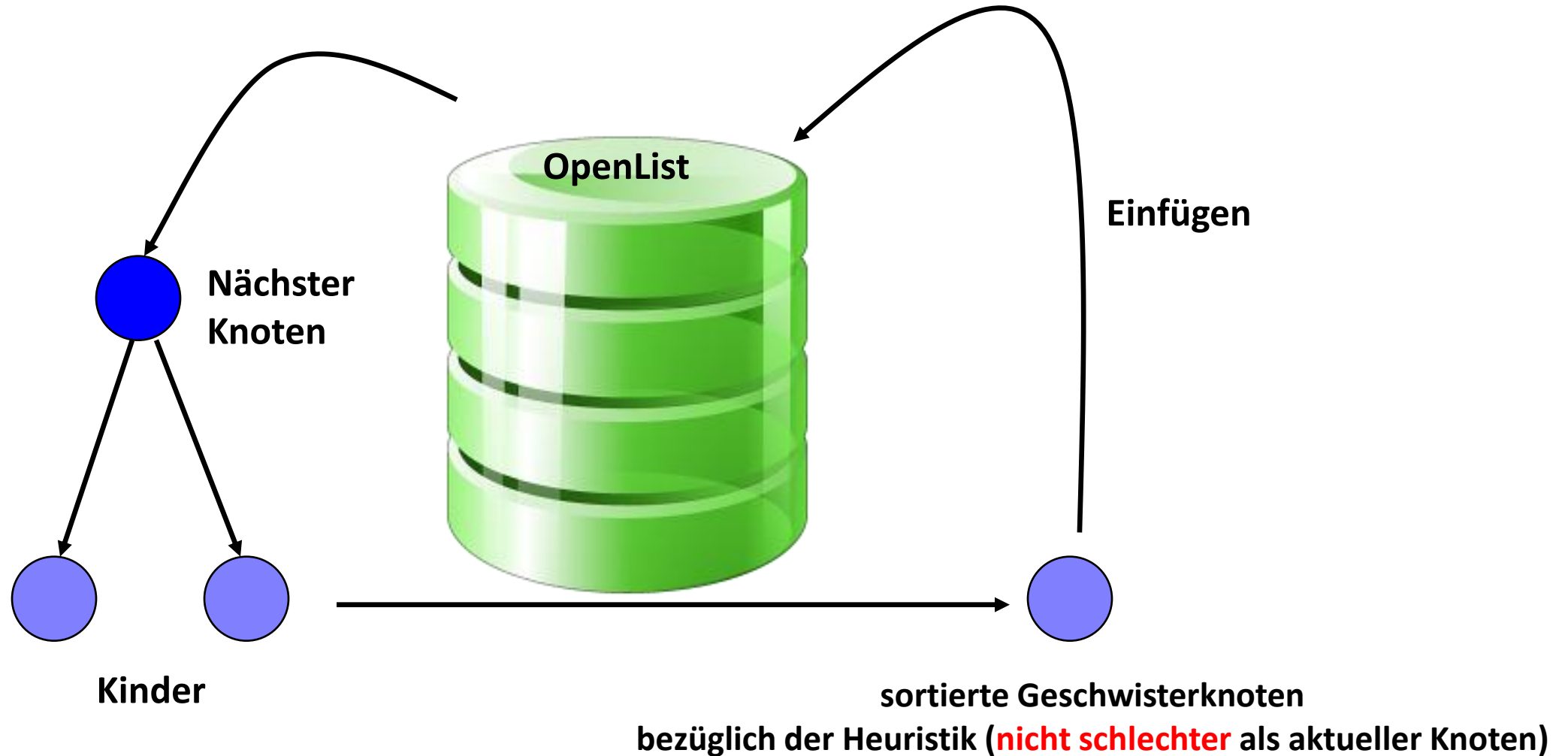


Hill Climbing (with Backtracking) – $f(n) = h(n)$

Hill Climbing – Bergsteigen – gierige lokale Suche – greedy local search

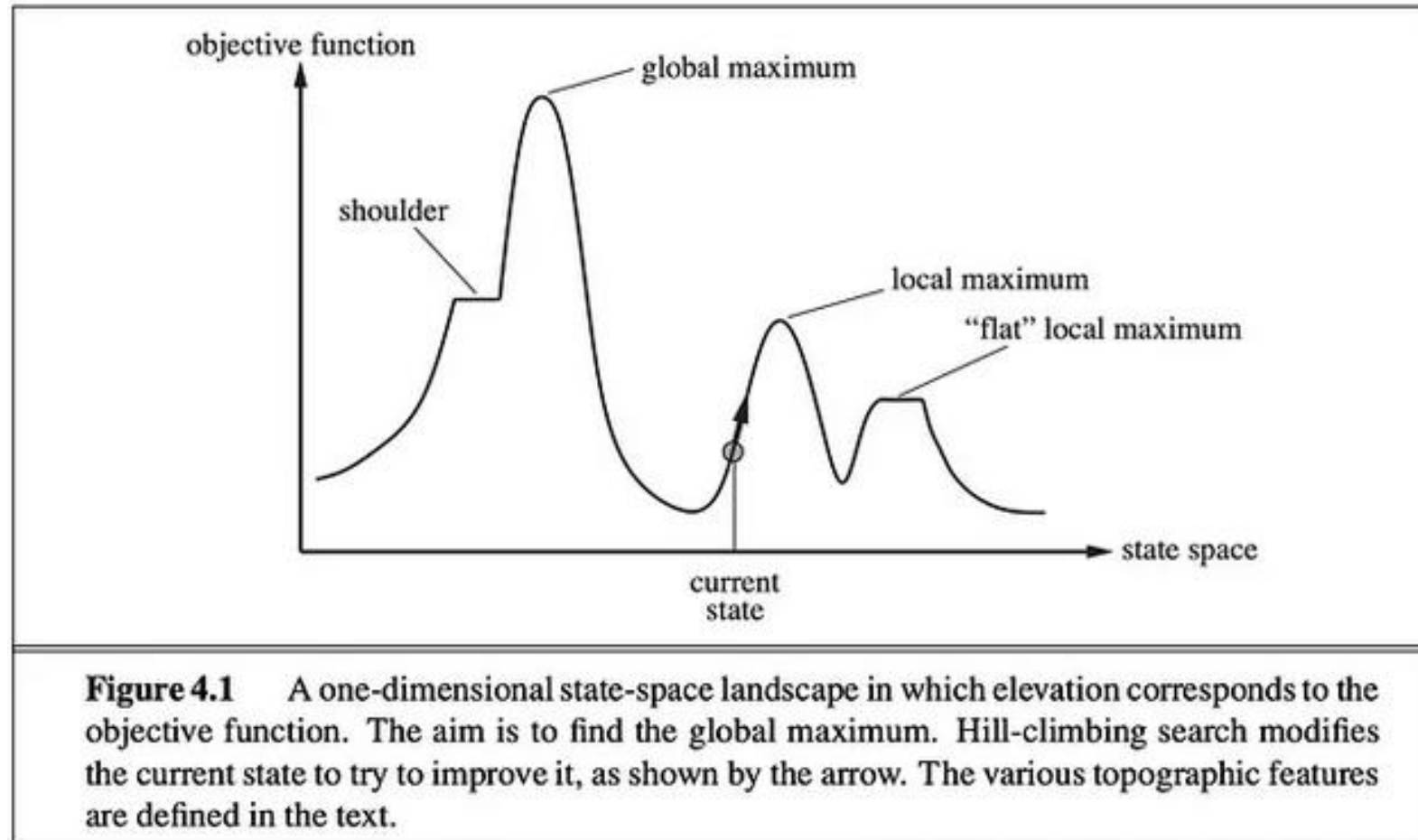
- Sonderform der Tiefensuche
- Nur direkte Nachfolger des aktuellen Knotens werden betrachtet.
 - Nachfolger mit geringstem Wert von h wird gewählt (steilster Anstieg).
 - Nachfolger mit Verschlechterung gegenüber aktuellem Zustand scheiden aus.
 - Häufig sehr effizient.
 - Extrem geringer Speicherbedarf (maximale Verzweigungsrate).
 - unvollständig.
 - Backtracking möglich
 - Problem: lokale Maxima.

Hill Climbing (with Backtracking)



Problem der lokalen Maxima, Plateaus, ... (1)

Der nächste Knoten, der expandiert wird, hat die kürzeste Entfernung zum Ziel.



Bildquelle: Russel, Stuart; Norvig, Peter: Artificial Intelligence: A Modern Approach. 2. Auflage, Prentice Hall, 2010.

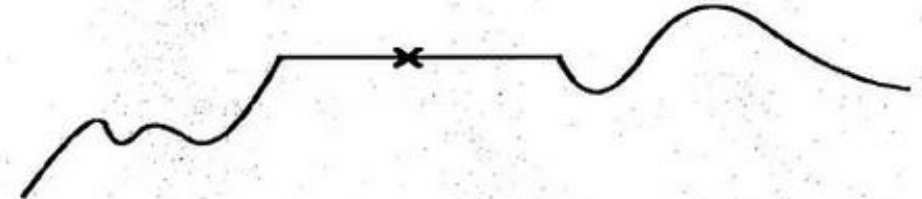
Problem der lokalen Maxima, Plateaus, ... (2)

- Heuristiken der Nachfolgeknoten sind schlechter der Bewertung des aktuellen Knotens.
- Es besteht aber die Möglichkeit, dass erst nach Durchlaufen eines schlechter bewerteten Knotens die Heuristiken wieder bedeutend besser werden.
- Des Weiteren Plateaus und Kammlinien möglich.

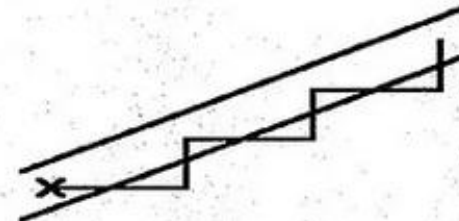
⊙ **Local maximum**



⊙ **Plateau**



⊙ **Ridge**



Hill Climbing Varianten

Simple Hill Climbing

...nimmt nur DEN Kinds-knoten mit dem steilsten Anstieg.

Steepest-Ascent Hill Climbing (Greedy Hill Climbing)

... nimmt alle möglichen Kinds-knoten und wählt den mit dem steilsten Anstieg.
(Hill Climbing mit Backtracking)

Stochastic Hill Climbing

...wählt zufällig eine Aufwärtsbewegung aus, wobei die Wahrscheinlichkeit davon abhängen kann, wie steil die Bewegung ist. Langsamer als die Wahl des steilsten Weges, kann aber in schwierigen Such-räumen bessere Lösungen finden, langsamer, weil es aus allen möglichen Aufwärtsbewegungen auswählt.

First-Choice Hill Climbing

... ist eine Variante des Stochastic Hill Climbing bei der zufällig Kinds-knoten generiert werden, bis einer gefunden wird, der besser als der aktuelle Zustand ist. Diese Methode wird verwendet, wenn es viele mögliche Züge gibt (Tausende), sodass es unpraktisch wäre, sie alle zu überprüfen.

Simulated Annealing

... versucht das globale Minimum in einem Optimierungsproblem zu finden. Veranschaulicht durch einen Ping-Pong-Ball auf einer unebenen Oberfläche: Im Ruhezustand bleibt er in der nächst-gelegenen Vertiefung (lokales Minimum) liegen. Durch Schütteln kann der Ball dem lokalen Minimum entkommen und möglicherweise das globale Minimum erreichen. Zu Beginn ist das Schütteln stark, so dass verschiedene Bereiche erkundet werden können. Mit der Zeit nimmt das Schütteln ab und der Algorithmus konzentriert sich allmählich darauf, die beste Lösung zu finden.

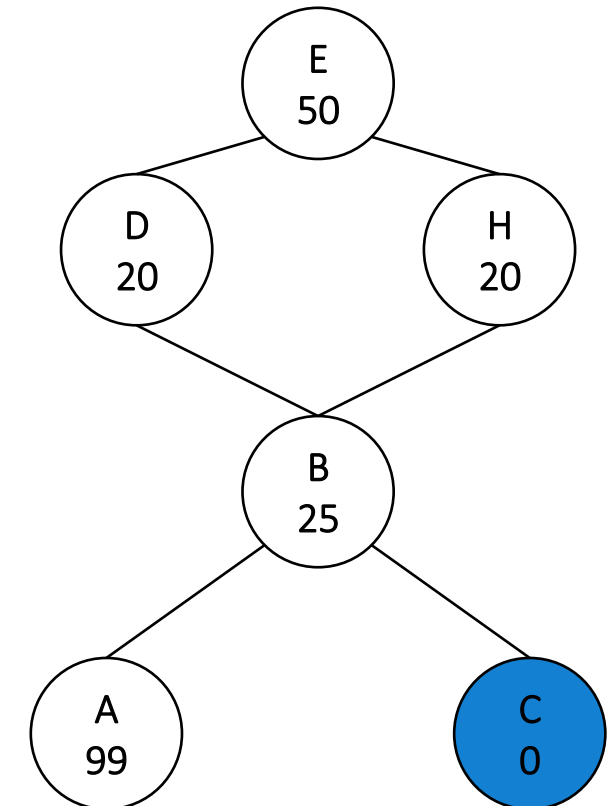
Random-Restart Approach

„Wenn du beim ersten Mal keinen Erfolg hast, versuche es erneut.“ lautet die Strategie dieses Ansatzes. Er beginnt mit zufälligen Anfangszuständen und führt Hill Climbing durch, bis er eine Lösung findet. Durch wiederholtes Neustarten erhöht er die Chancen, das globale Optimum zu finden. Möglicherweise findet er sogar den Zielzustand selbst.

Überlegungen Hill Climbing with Backtracking

- Wie funktioniert der Algorithmus, wenn es mehrere gleich gut bewertete nächste Schritte gibt und keine Quantifizierung vorliegt?
- Wie wird sich der zurückgelegte Weg gemerkt (besuchte Knoten, Pfad)?
- Was passiert, wenn es sich nicht um einen Baum, sondern einen Graphen handelt?

Schritt	OpenList
1	E(50)
2	D(E,20), H(E,20)
3	H(E,20)
4	lokales Maximum



Hill Climbing mit Backtracking vs. Bestensuche (1)

Hill Climbing mit Backtracking

- Informierte Tiefensuche (aber ohne Berücksichtigung von g).
- Aber: Geschwister-Knoten jeweils untereinander sortiert.
- Backtracking.
- Nicht vollständig.

Bestensuche

- Informierte Breitensuche (aber ohne Berücksichtigung von g).
- Alle Knoten entsprechend h sortiert.
- Ebenfalls nicht vollständig!

Übung Hill Climbing m. B.

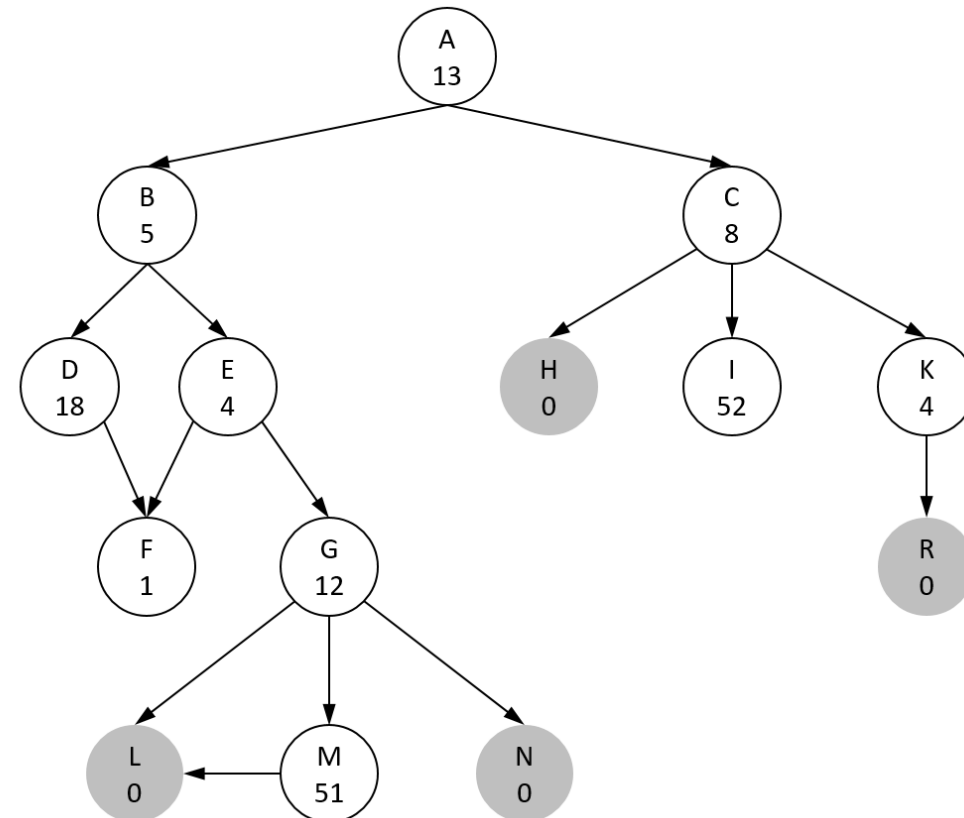
Gegeben ist folgender Suchbaum. Ermitteln Sie OpenList, Reihenfolge der besuchten Knoten und den Pfad zum ersten gefundenen Ziel (außer der Algorithmus terminiert). Ziel: ●

Vereinbarungen:

- Quantifizierung
Buchstaben, die weiter vorn im Alphabet stehen, werden bei gleicher Bewertung bevorzugt

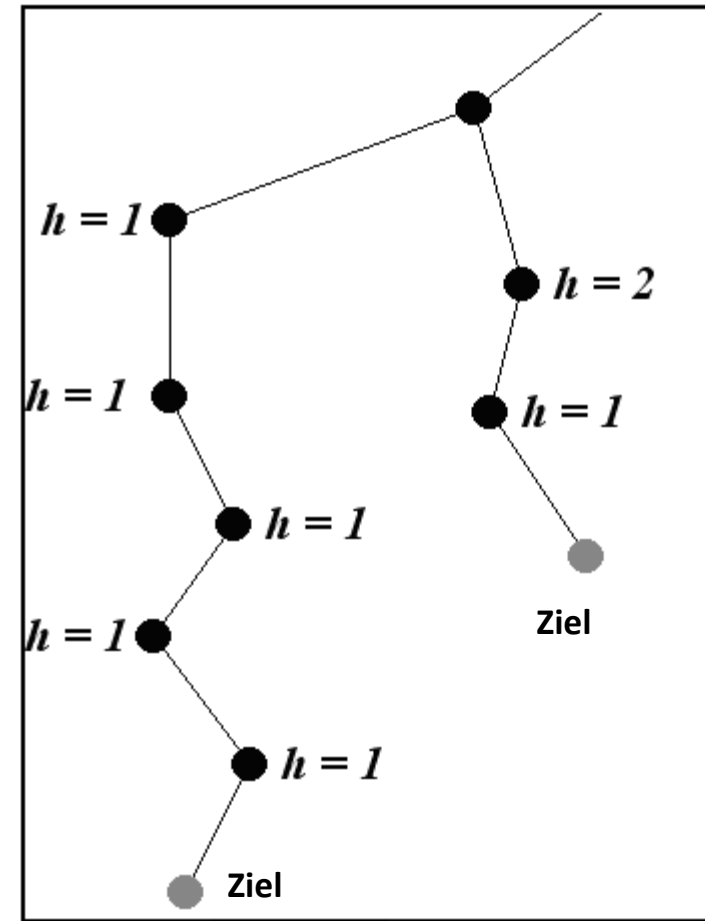


10 Minuten



Probleme durch Vernachlässigung der bisherigen Kosten (Funktion g)

- Optimale Lösung kann verfehlt werden
- Lösung insgesamt kann verfehlt werden (Verlust der Vollständigkeit)
- unendlich lange Pfade mit guter Bewertung h



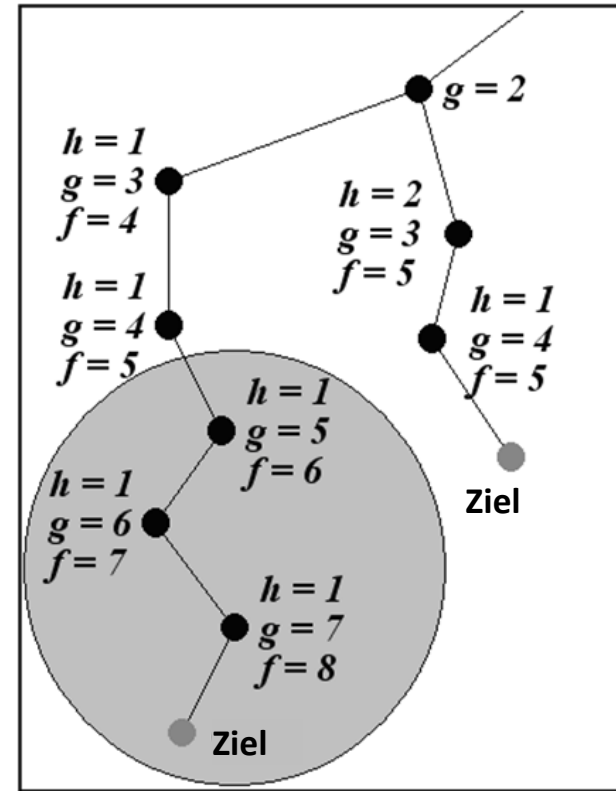
A / A*-Algorithmus

A/A*-Algorithmus – $f(n) = g(n) + h(n)$

Gesamtkosten eines Knotens $f(n) =$

+ bisherige Kosten ab dem Start $g(n)$
+ geschätzte Restkosten bis zum Ziel $h(n)$

$$f(n) = g(n) + h(n)$$



A/A*-Algorithmus – Beispiel (1)

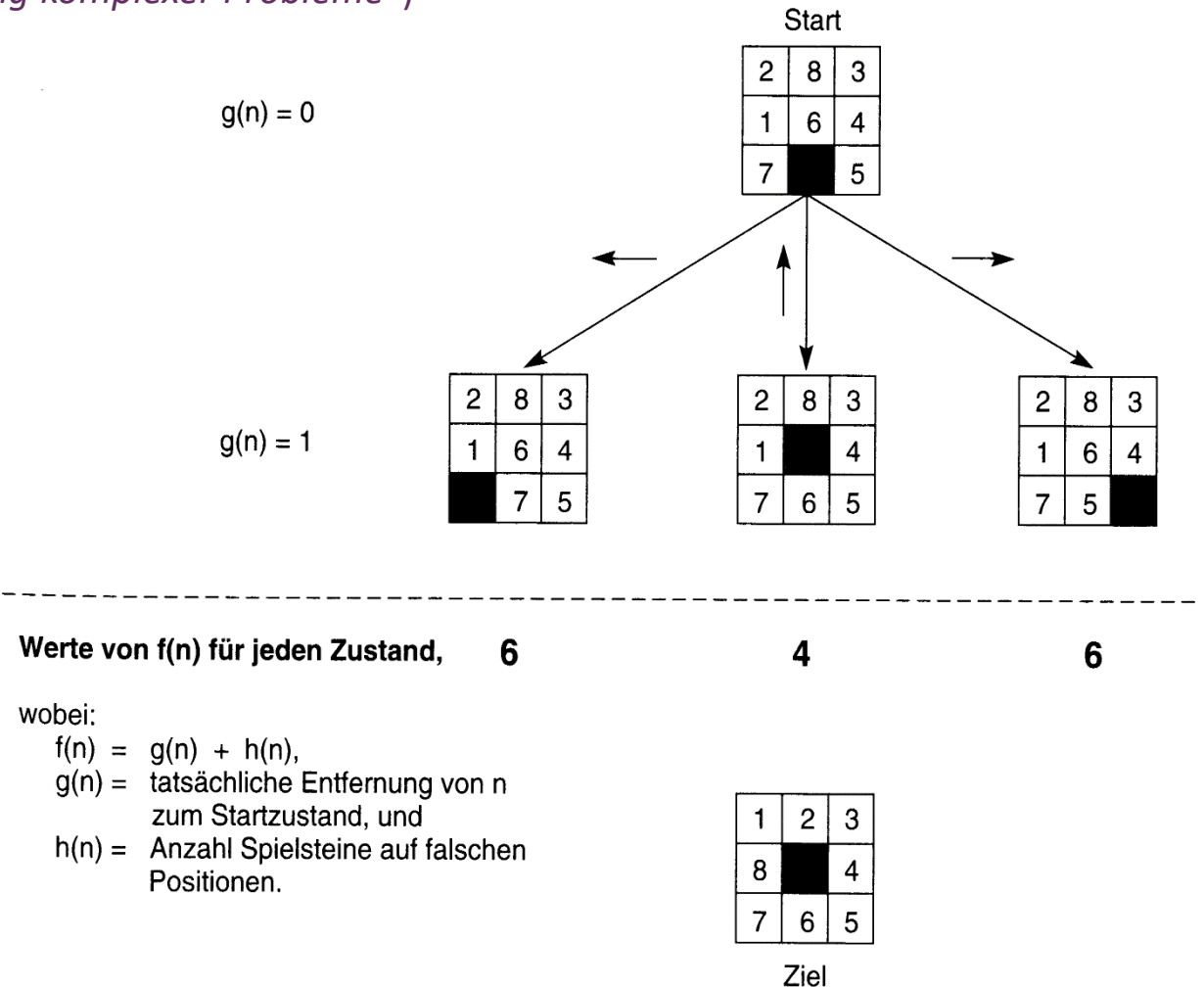
(aus Luger: „Künstliche Intelligenz – Strategien zur Lösung komplexer Probleme“)

Beispiel: 8-Puzzle mit Heuristik h_1

(Anzahl Spielsteine auf falscher Position)

$g(n) = 0$

$g(n) = 1$



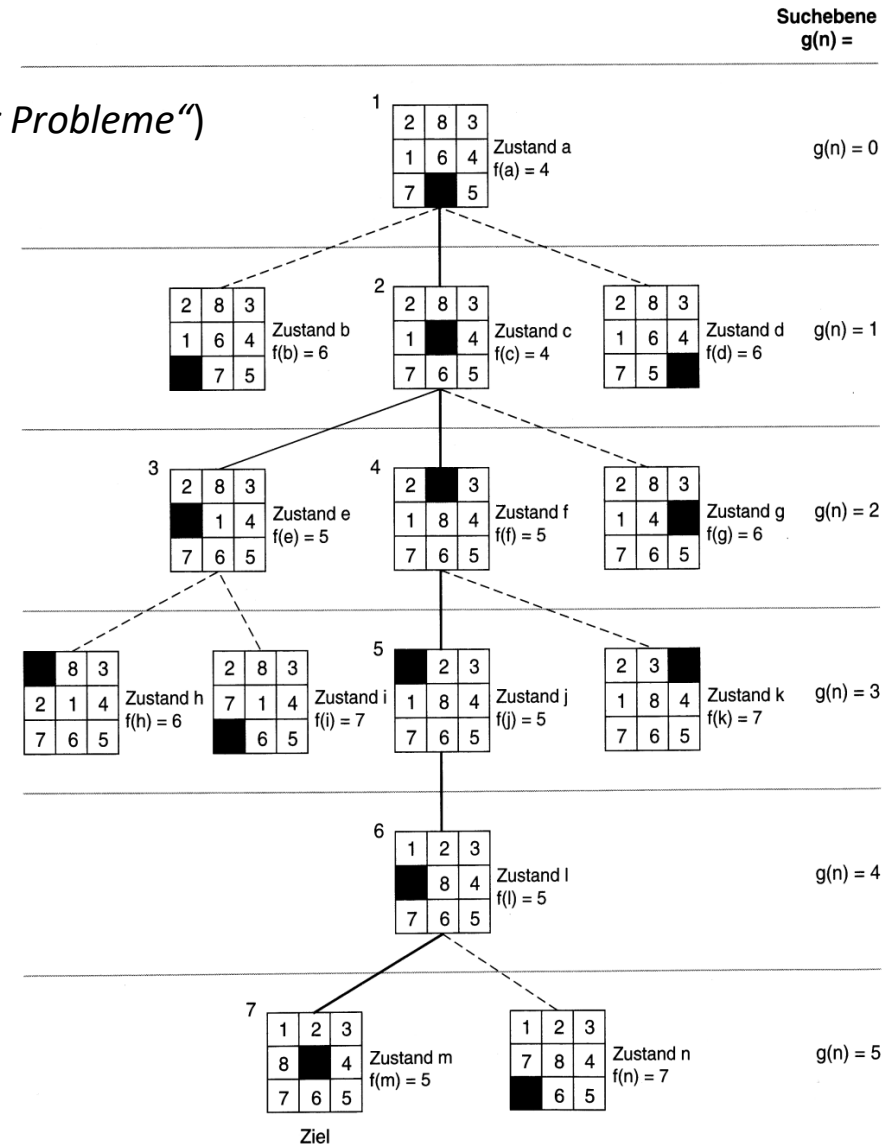
A/A*-Algorithmus – Beispiel (3)

(aus Luger: „Künstliche Intelligenz – Strategien zur Lösung komplexer Probleme“)

Beispiel:

- 8-Puzzle mit Heuristik h_1 ,
- vollständig bis zum Ziel expandierter Suchbaum

Das Ziel hat einen
Kostenfaktor von 5.



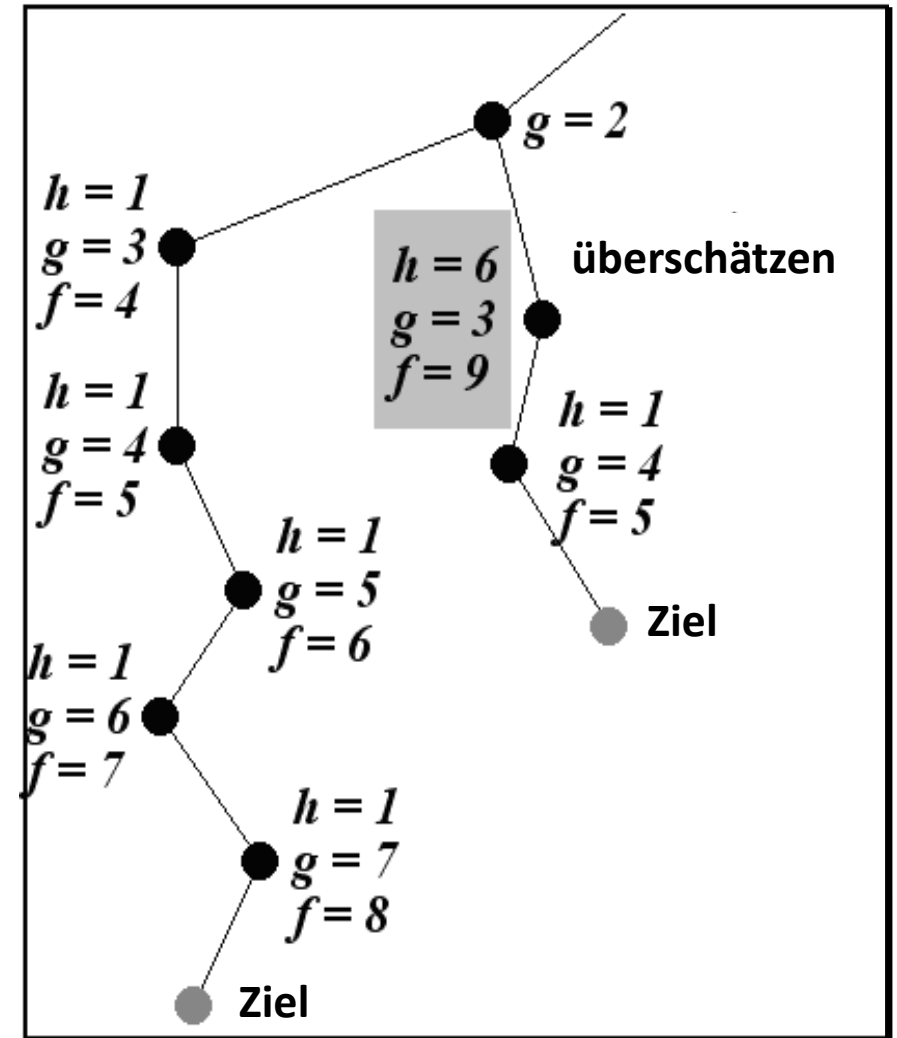
A- und A*-Algorithmus – Probleme mit der Heuristik

Findet der A-Algorithmus die optimale Lösung?

- nicht unbedingt,
- wenn heuristische Funktion h die Kosten bis zum Ziel überschätzt, kann ein anderes, nicht optimales Ziel gefunden werden.



In diesem Fall spricht man vom A-Algorithmus und nicht vom A*-Algorithmus.



Eigenschaften von Heuristiken

Schätzfunktion / Heuristik h heißt

- **zulässig**, wenn sie die Restkosten nicht überschätzt:
 $g(\text{Zielknoten}) - g(n) \geq h(n)$, für alle n , bzw.
 $h^*(n) \geq h(n)$, für alle n ,
(h^* : Funktion der tatsächlichen Kosten)
- **monoton** (konsistent), wenn die Kosten zwischen **zwei beliebigen Knoten** nie überschätzt werden
D.h.: Die Werte von f entlang eines Pfades dürfen nicht fallen.
Bsp.: Für unser 8-Puzzle: h_1 und h_2 sind monoton,
Routenplaner: Luftlinie ist monoton

Zulässige Heuristiken sind meistens auch monoton (Gegenbeispiele schwer zu finden).

Für zwei zulässige Funktionen h_1, h_2 gilt:

Wenn $h_2(n) \geq h_1(n)$ für alle n , dann ist h_2 informierter.

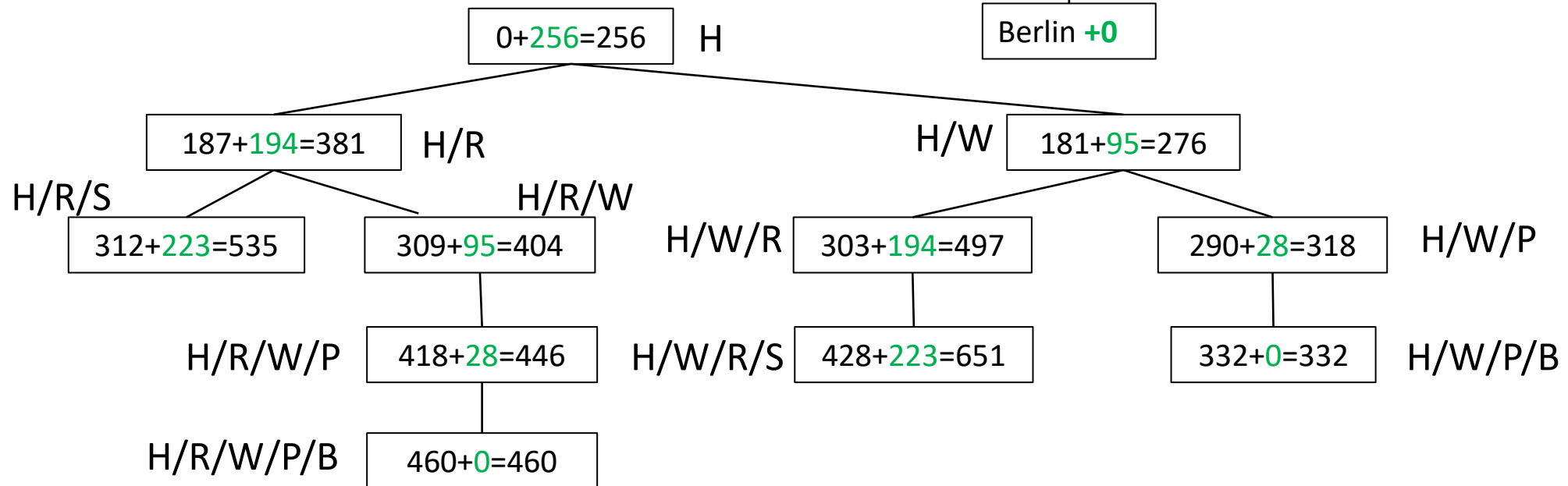
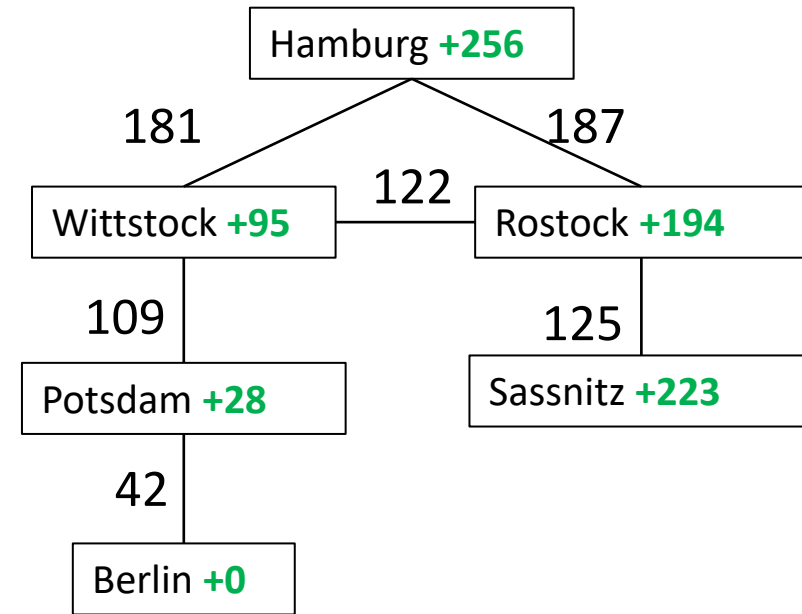
Bsp.: 8-Puzzle: h_2 ist informierter als h_1

A*-Algorithmus (1)

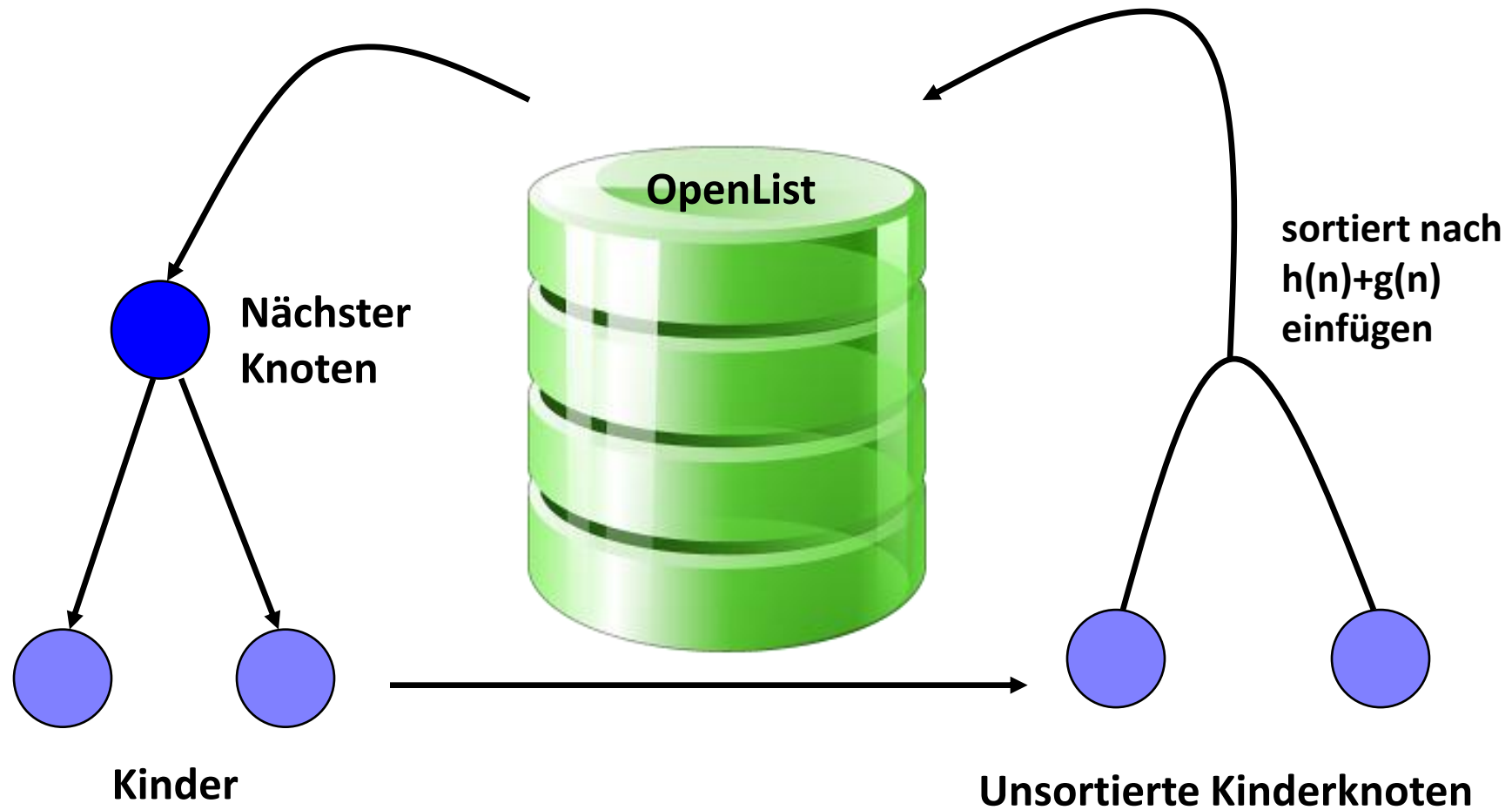
- Wenn die in einem A-Algorithmus verwendete Heuristik zulässig ist, dann bezeichnet man dies als **A*-Algorithmus**.
- Der A*-Algorithmus ist optimal, d.h. er findet den günstigsten Pfad von einem Startzustand zu einem Zielzustand.

A*-Algorithmus (2)

- Kosten Stadt1-Stadt2:
- Streckenlänge der Straßenverbindung.
- **Heuristik:**
Luftlinienentfernung zum Ziel



A*-Algorithmus (3)

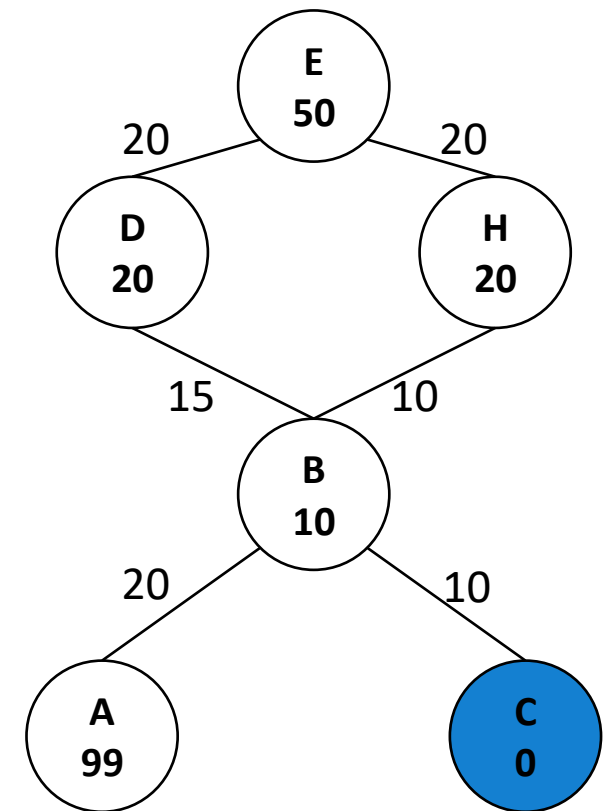


Überlegungen A* - Multi-Path-Pruning

- Wie funktioniert der Algorithmus, wenn es mehrere gleich gut bewertete nächste Schritte gibt und keine Quantifizierung vorliegt?
- Was passiert, wenn es sich nicht um einen Baum, sondern einen Graphen handelt?
- A/ A* verwendet **Multiple Path Pruning (MPP)**!

Schritt	OpenList
1	E(50)
2	D(40), H(40)
3	H(40), B(45)
4	B(40)
5	C(40), A(149)
6	Ziel gefunden , A(149)

- Pfad zum Ziel: EHBC
- besuchte Knoten: EDHBC
- Backtracking möglich.



Überlegungen A* - Multi-Path-Pruning

- Wie funktioniert der Algorithmus, wenn es mehrere gleich gut bewertete nächste Schritte gibt und keine Quantifizierung vorliegt?

- Was passiert, wenn es sich nicht um einen Baum, sondern einen Graphen handelt?

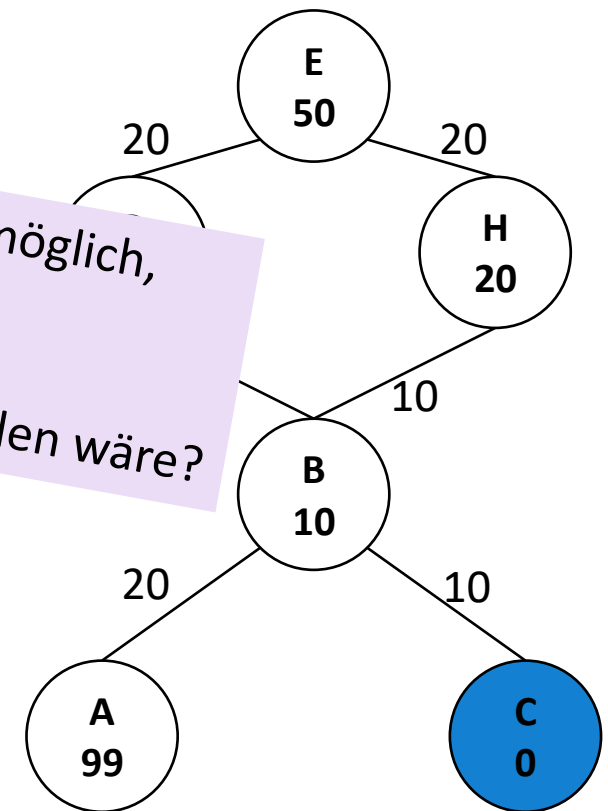
Multi-Path Pruning (MPP)!

Die Ersetzung des Knotens B mit seiner neuen Bewertung ist hier nur möglich, weil B sich noch in der OpenList befindet!

Was, wenn B bereits bearbeitet worden und aus der OpenList entfernt worden wäre?

3	A(149)
4	B(40)
5	C(40), A(149)
6	Ziel gefunden , A(149)

- Pfad zum Ziel: EHBC
- besuchte Knoten: EDHBC
- Backtracking möglich.



ClosedList

- Um Informationen über bereits abgearbeitete Knoten nicht zu verlieren, wird zusätzlich zur OpenList eine ClosedList benötigt.
- Dies geht natürlich zu Lasten des benötigten Speicherplatzes.
- Welche Informationen zu den einzelnen Knoten in der ClosedList gespeichert werden, ist implementationsabhängig.

Z. Bsp:

- eigene Bewertung
- Link zum günstigsten Vorgängerknoten

A*-Algorithmus – Bewertung Aufwand

- Für die meisten Probleme exponentiell
 - Anzahl der expandierten Knoten wächst exponentiell in Abhängigkeit von der Tiefe bis zum Zielknoten
- Primäres Problem: Speicherbedarf
- Verbesserung durch Modifikation des Algorithmus
 - Weniger Speicherbedarf zu Lasten der Rechenzeit.
 - Nach wie vor optimal und vollständig.

A*-Algorithmus – verwandte Algorithmen

verwandte Algorithmen:

- Algorithmus von Dijkstra (verwendet keine Heuristik),
- Greedy Algorithmus (nur Restkosten werden betrachtet).

verbesserte Algorithmen:

- IDA* (Iterative Deepening A*):
Variante der iterativen Tiefensuche.
- RBFS (Recursive Best-First Search):
beschränkt den Speicherplatzverbrauch linear zur Länge der Lösung.
- MA* (Memory-Bounded A*) und SMA* (Simplified MA*):
benutzen jeweils eine fest vorgegebene Speicherplatzmenge.

Erweiterung:

- D* (dynamischer A*):
wiederholte Neuplanung nach Erhalt neuer Informationen
(Navi: Brücke ist defekt, Roboter in Katastrophengebieten, ...)

Übung (1) A/A*

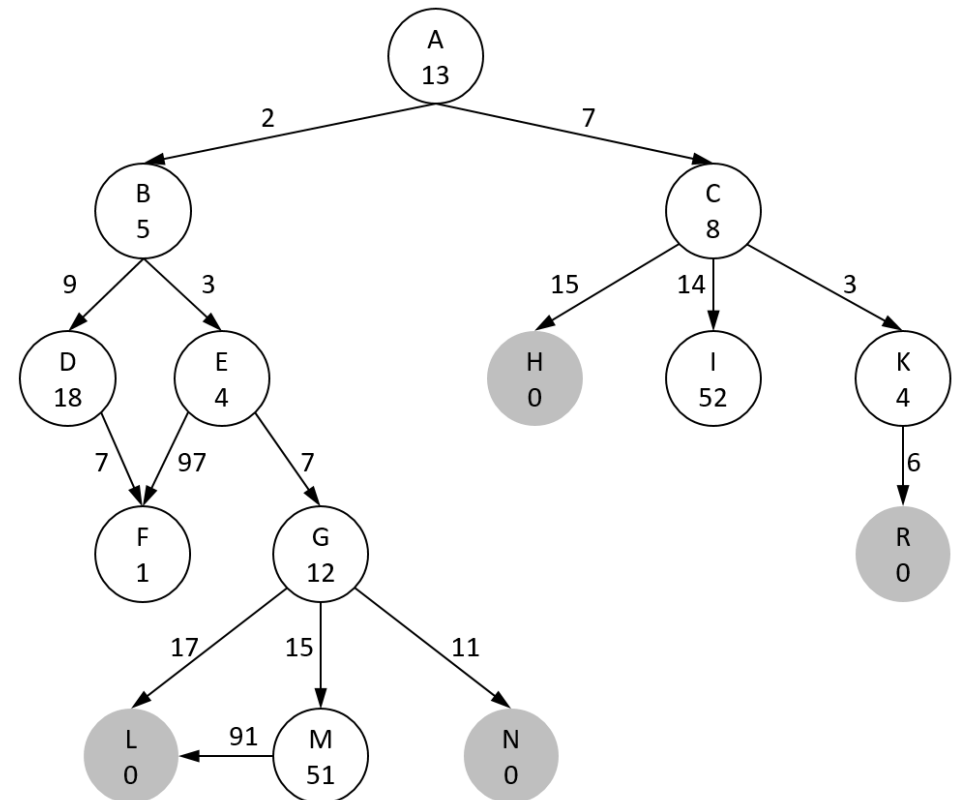
Gegeben ist folgender Suchbaum. Ermitteln Sie OpenList, ClosedList, Reihenfolge der besuchten Knoten und den Pfad zum ersten gefundenen Ziel (außer der Algorithmus terminiert). Ziel: ●

Vereinbarungen:

- Quantifizierung
Buchstaben, die weiter vorn im Alphabet stehen, werden bei gleicher Bewertung bevorzugt

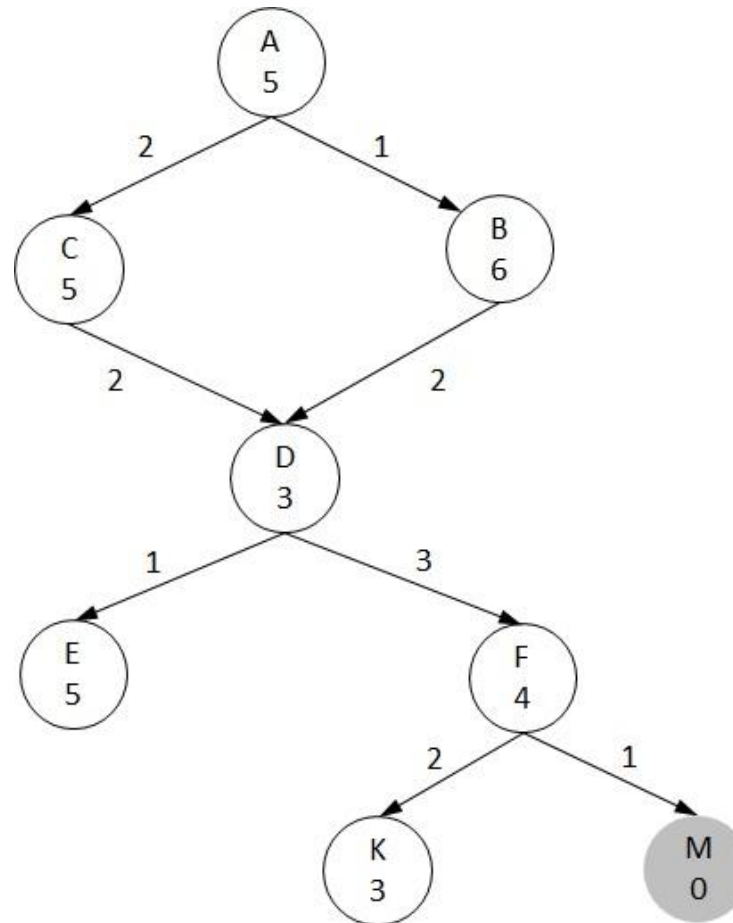


10 Minuten



Übung (2) A/A*

Gegeben ist folgender Suchbaum. Ermitteln Sie OpenList, ClosedList, Reihenfolge der besuchten Knoten und den Pfad zum Ziel (außer der Algorithmus terminiert). Ziel: ●



10 Minuten

Suchverfahren – Aufgaben Nutzer / Suchverfahren

Nutzer

muss folgende Informationen bereitstellen:

- Geeignete formale **Darstellung für Zustände**
- **Startzustand, Zielzustände**
- Korrekte **Zustandsübergänge**
- **g** – Kosten für EINEN Zustandsübergang
- **h** – geschätzte Entfernung eines Zustands vom Ziel

Suchprogramm

übernimmt die Aufgaben:

- **Verwaltung** der Kandidaten
- Auswahl des **nächsten Kandidaten**
- **Summieren** der bisherigen Kosten
- **Berechnung** von $g+h$
- **Cycle checking**

Übersicht Suchverfahren

Verfahren		Charakteristika	Backtracking
Breitensuche	-	Ältere Kandidaten werden vor neueren behandelt.	ja
Tiefensuche	-	Neuere Kandidaten werden vor älteren bearbeitet.	ja
Nearest Neighbour	g	Es wird nur genau der Nachfolgeknoten, der die geringsten Kosten g verursacht, weiter betrachtet.	nein
Bestensuche	h	Alle Kandidaten werden nach ihrer Heuristik sortiert abgearbeitet.	ja
Hill Climbing	h	Es wird der Nachfolgeknoten mit der besten h-Bewertung weiter betrachtet, wobei die h-Bewertung nicht schlechter als die h-Bewertung des Vaterknotens sein darf.	nein
Hill Climbing with Backtracking	h	Es werden die Nachfolgeknoten nach ihrer h-Bewertung sortiert vor allen anderen Kandidaten weiter betrachtet, wobei die h-Bewertung nicht schlechter als die h-Bewertung des Vaterknotens sein darf.	ja
A / A* Algorithmus	g+h	Bestensuche mit Bewertungs-Funktion g+h, Mehrfachwege werden vermieden. Bei A*: h darf nicht überschätzen!	ja

Multi-Path-Pruning auch bei anderen Suchalgorithmen?

- um auch bereits abgearbeitete Knoten zu beachten, muss eine ClosedList geführt werden
- diese benötigt zusätzlichen Speicherplatz
 - Breitensuche benötigt an sich schon den meisten Speicherplatz
 - Tiefensuche ist gerade interessant, weil sie den geringsten Speicherplatz benötigt
 - Hill-Climbing mit Backtracking – wird für Optimierungsprobleme genutzt
 - Bestensuche, ist gerade interessant, weil sie geringen Speicherplatz benötigt
- um Knoten nicht mehrfach in der OpenList einzutragen, können oben genannte Suchalgorithmen dahingehend erweitert werden, dass sie Doppelungen ausschließen, das geschieht auf Implementationsebene.

Informierte Suche – Lernziele

- ... Aufbau eines allgemeinen Suchalgorithmus erläutern können,
- ... Eigenschaften, Vor- und Nachteile und Arbeitsweise der vorgestellten Suchalgorithmen erläutern und sie untereinander vergleichen können,
- ... Heuristiken erstellen, verstehen und bewerten können,
- ... Knoten anhand eines Suchalgorithmus expandieren können,
- ... erklären können, wie Steuerung der Suchstrategie über Sortierung der OpenList für die verschiedenen Suchverfahren erfolgt,
- ... für ein Problem das geeignetste Suchverfahren auswählen und anwenden können.